

PHP Conditionals

Contents

1 Chapter 3: Conditionals in PHP	1
1.1 Understanding if, else, and elseif Statements	1
1.1.1 Basic if Statement	1
1.1.2 The else Clause	2
1.1.3 Using elseif for Multiple Conditions	3
1.2 The switch Statement	4
1.2.1 Basic switch Syntax	4
1.2.2 Conclusion	4
2 Conditionals in PHP 8	5
2.1 Alternative Syntax for Conditionals	5
2.1.1 Alternative Syntax for if, elseif, else	5
2.1.2 Alternative Syntax for switch	5
2.2 Match Expression (PHP 8)	6
2.2.1 Basic Syntax	6
2.2.2 Features	6
2.2.3 Example	6
2.2.4 When to Use match	6
2.2.5 Conclusion	6

1 Chapter 3: Conditionals in PHP

Welcome to Chapter 3 of my book, designed to guide you through learning PHP. In this chapter, we will explore conditionals, an integral part of programming that allows you to make decisions based on different conditions. Conditionals are crucial for creating dynamic and interactive web pages, enabling responses to user inputs, data changes, or other events.

1.1 Understanding if, else, and elseif Statements

Conditionals in PHP begin with the `if` statement, the most fundamental form of conditional logic. It allows for the execution of a block of code only if a certain condition is true.

1.1.1 Basic if Statement

The syntax for an if statement is straightforward:

```
<?=  
    if (condition) {  
        // Code to be executed if condition is true  
    }
```

For example, the following code checks if a variable 'age' is greater than 18:

```
<?=  
$age = 20;  
  
if ($age > 18) {  
    echo "You are an adult.";  
}
```

For instance, if you want to display a message only when a user is logged in, you might use:

```
<?=  
    // Assume $logged_in is a variable that stores true or false  
    if ($logged_in) {  
        echo "Welcome, you are logged in!";  
    }
```

The code within the curly braces {} will execute if the condition within the parentheses () is true. In this case, \$logged_in is a boolean variable that indicates the user's login status.

1.1.2 The else Clause

An else clause can be added to specify what should happen when the if condition is false. The syntax as follows>

```
<?=  
    if (condition) {  
        // Code to be executed if condition is true  
    } else {  
        // Code to be executed if condition is false  
    }
```

The following code checks if a variable 'score' is greater than 90:

```
<?=  
$score = 95;  
  
if ($score > 90) {  
    echo "You scored an A.";  
} else {  
    echo "You scored below an A.";  
}
```

This code checks if \$logged_in is true and welcome us, or if not tell us to log in.

```
<?=  
    if ($logged_in) {
```

```

    echo "Welcome, you are logged in!";
} else {
    echo "Please log in to access this page.";
}

```

Here, if `$logged_in` is false, the message "Please log in to access this page." will be displayed.

1.1.3 Using elseif for Multiple Conditions

The `elseif` statement is used to add multiple conditions to an `if` statement. It is similar to the `else` statement, but it allows you to check for multiple conditions. The syntax is as follows:

```

<?=
    if (condition1) {
        // Code to be executed if condition1 is true
    } elseif (condition2) {
        // Code to be executed if condition1 is false and condition2 is true
    } else {
        // Code to be executed if conditions1 and condition2 are false
    }

```

Example 1:

```

<?=
    // Assume $age is a positive integer
    if ($age < 13) {
        echo "You are a child.";
    } elseif ($age < 18) {
        echo "You are a teenager.";
    } elseif ($age < 65) {
        echo "You are an adult.";
    } else {
        echo "You are a senior.";
    }

```

Example 2: the following code checks if a variable 'grade' is A, B, or C:

```

<?=
$grade = "A";

if ($grade == "A") {
    echo "Your grade is an A.";
} elseif ($grade == "B") {
    echo "Your grade is a B.";
} else {
    echo "Your grade is a C.";
}

```

`elseif` clauses are evaluated in order and only one block of code is executed: the first one where the condition is true. If none are true, and there's an `else` clause, its code will run.

1.2 The switch Statement

Another powerful conditional in PHP is the `switch` statement, which allows for actions based on different values of a variable or expression (check for multiple conditions from a set of possible values).

1.2.1 Basic switch Syntax

The structure of a `switch` statement is as follows:

```
<?=  
    switch ($variable) {  
        case value1:  
            // Code to be executed if $variable equals value1  
            break;  
        case value2:  
            // Code to be executed if $variable equals value2  
            break;  
        ...  
        default:  
            // Code to be executed if $variable is different from all values  
    }
```

For example:

```
<?=  
    // Assume $color is a string variable  
    switch ($color) {  
        case "red":  
            echo "You like red.";  
            break;  
        case "blue":  
            echo "You like blue.";  
            break;  
        case "green":  
            echo "You like green.";  
            break;  
        default:  
            echo "You don't have a favorite color.";  
    }
```

In the `switch` statement, each case is compared with the value of `$color`, and the corresponding code block is executed if there's a match. The `break` statement prevents the code in subsequent cases from running.

1.2.2 Conclusion

Conditionals are a fundamental aspect of PHP programming, enabling decision-making based on various conditions. Through `if`, `else`, and `elseif` statements, you can execute different code blocks

based on boolean conditions. Meanwhile, switch statements offer a streamlined method for handling multiple potential values of a variable.

Having mastered these concepts, you're now equipped to add more interactive and responsive elements to your PHP projects. In the next chapter, we'll dive into loops, a feature that allows for repetitive actions, further expanding your PHP programming toolkit.

2 Conditionals in PHP 8

In PHP 8, you can use alternative syntax for conditionals and the new Match expression, which offers a more concise and readable way of handling multiple conditional checks. Let's explore both.

2.1 Alternative Syntax for Conditionals

PHP provides an alternative syntax for `if`, `elseif`, `else`, and `switch` statements, which is particularly useful in templates and HTML embedded PHP scripts.

2.1.1 Alternative Syntax for `if`, `elseif`, `else`

Instead of using curly braces `{ }`, you use `:` to start and `endif;` to end the conditional block.

```
<?php if ($condition): ?>
    <!-- HTML to display if condition is true -->
<?php elseif ($anotherCondition): ?>
    <!-- HTML to display if another condition is true -->
<?php else: ?>
    <!-- HTML to display if all conditions are false -->
<?php endif; ?>
```

2.1.2 Alternative Syntax for `switch`

Similar to `if` statements, `switch` can use `endswitch;` in place of curly braces.

```
<?php
switch ($variable):
    case "value1":
        // Code for value1
        break;
    case "value2":
        // Code for value2
        break;
    //...
    default:
        // Default code
endswitch;
?>
```

2.2 Match Expression (PHP 8)

The `match` expression introduced in PHP 8 is similar to `switch`, but it's more powerful and concise. It returns a value and doesn't require a `break` statement.

2.2.1 Basic Syntax

```
$result = match ($variable) {  
    'value1' => 'Result for value1',  
    'value2' => 'Result for value2',  
    default => 'Default result',  
};
```

2.2.2 Features

- **Value Return:** Directly returns the value of the matched case.
- **Expression Comparison:** Compares values as expressions, not just simple comparisons.
- **Strict Type Checking:** Uses strict type comparison (===).

2.2.3 Example

```
$color = "red";  
$message = match ($color) {  
    "red" => "The color is red",  
    "green" => "The color is green",  
    "blue" => "The color is blue",  
    default => "Unknown color",  
};  
  
echo $message;
```

This will output: The color is red.

2.2.4 When to Use `match`

- **Simple Value Mapping:** When you need to map specific values to results.
- **Concise Conditional Logic:** Reduces the verbosity of multiple `if-elseif` statements.
- **Strict Comparison:** Automatically applies strict comparison, making it more reliable for certain types of data.

2.2.5 Conclusion

The alternative syntax for conditionals and the Match expression in PHP 8 provide cleaner and more concise ways to write conditional logic. They are particularly useful in scenarios where readability and maintainability of code are priorities, such as in template files or when handling many conditions.